

SIMATS ENGINEERING

ASSESSMENT TOOL 3

COURSE NAME: Artificial Intelligence

COURSE CODE: CSA17

COURSE OUTCOME COVERED

C05: *Design AI-based systems using PySpark, RDD operations, and intelligent agent architectures, using scalable systems and integrate components in distributed AI applications. (BTL 5)*

Assessment Tool Weightage: 50%

QUESTIONS: 5

Duration : 1 Hour
Total Marks:50

Annexure A

Technical Presentation

Code of Conduct:

*I **B.Bhargavi 192311136**(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:*B.Bhargavi*

Technical Presentation

1.ReactiveAgent-Based Traffic Control System

Prepare a technical presentation on designing a Reactive Agent for a smart traffic control system. Explain how the agent responds to real-time traffic inputs, manages signal changes, and ensures minimal congestion using streaming data processing.

2. Goal-Based Agent for Healthcare Diagnosis

Develop a presentation on a Goal-Based Agent used in an AI healthcare system. Describe how the agent sets diagnostic goals, evaluates patient data, and selects optimal treatment pathways using decision-making logic and data pipelines.

3. Utility-Based Agent for E-Commerce Recommendations

Create a technical presentation on a Utility-Based Agent that generates personalized product recommendations. Explain how the agent assigns utility values to different options and optimizes user satisfaction using large-scale data processing.

4. Learning Agent for Personalized Education System

Present the design of a Learning Agent in an AI-driven educational platform. Explain how the agent improves performance over time using feedback, adapts content difficulty, and integrates with distributed data processing systems like PySpark.

5. Multi-Agent System for Disaster Management

Prepare a presentation on a Multi-Agent System where multiple agents (sensor agents, decision agents, communication agents) collaborate for disaster detection and response. Explain coordination strategies, data sharing, and real-time decision-making.

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Technical Content Accuracy	30	Highly accurate, in-depth, and insightful technical explanation	Technically correct content with adequate depth and clarity	Basic concepts presented with limited accuracy and depth	Content contains major technical errors; concepts poorly understood
Problem Identification	25	Problem rigorously analyzed using appropriate and advanced methods	Problem clearly defined with a logical engineering approach	Problem identified but analysis or methodology is weak	Problem statement unclear or incorrect; no logical approach
Use of Tools	20	Advanced tools, well-analyzed data, and highly effective visuals	Appropriate tools and data used effectively with clear visuals	Limited use of tools and basic data interpretation	Minimal or incorrect use of tools and data; poor visuals
Communication	15	Highly engaging, confident, and audience-focused delivery	Clear, organized, and professional communication	Understandable but lacks clarity, structure, or confidence	Poor organization, unclear speech, ineffective slides
Professionalism	10	Exemplary professionalism, leadership, and ethical awareness	Professional conduct with effective team collaboration	Basic professionalism; uneven team participation	Lacks professionalism; minimal contribution or ethical awareness

1. Reactive Agent-Based Traffic Control System

Introduction

A Reactive Agent-Based Traffic Control System is an intelligent traffic management system in which an agent observes current traffic conditions and immediately responds to changes. The agent does not depend mainly on long-term planning; instead, it uses real-time sensor and streaming data to make rapid signal-control decisions.

Objectives

- Monitor traffic density and vehicle flow in real time.
- Detect congestion, queues, accidents, and unusual traffic conditions.
- Automatically adjust traffic signal timing.
- Reduce waiting time, queue length, and fuel consumption.
- Provide continuous and fast responses using streaming data processing.

System Architecture

The system consists of traffic sensors or cameras, a streaming-data layer, a reactive agent, a signal-control module, and monitoring dashboards. Sensors continuously generate traffic information. The streaming layer processes incoming events, while the reactive agent applies predefined rules to select the appropriate signal action.

Working Principle

1. Sensors collect vehicle count, speed, lane occupancy, and queue-length information.
2. The streaming platform receives and processes the incoming data continuously.
3. The reactive agent compares current traffic conditions with predefined thresholds.
4. If congestion is detected, the agent changes green-light duration or signal priority.
5. The updated signal command is sent to the traffic controller.
6. The agent continuously observes the new traffic state and repeats the process.

Reactive Decision Logic

A simple rule-based decision can be expressed as: IF traffic density is high, THEN increase green time for that direction; IF density is low, THEN reduce green time; IF an emergency vehicle is detected, THEN provide priority to the required lane. These rules allow the system to respond within seconds.

Streaming Data Processing

Streaming technologies such as Apache Kafka, Apache Spark Streaming, or similar event-processing systems can be used to handle continuous traffic events. The system can calculate moving averages, detect sudden traffic increases, and forward the latest state to the reactive agent.

Advantages

- Fast response to changing traffic conditions.
- Reduced congestion and average waiting time.
- Better utilization of road capacity.
- Continuous monitoring without manual intervention.
- Scalable architecture for multiple intersections.

Conclusion

A reactive traffic agent provides a practical approach to smart traffic control because it can make immediate decisions from live data. By combining sensors, streaming processing, and rule-based actions, the system can reduce congestion and improve traffic-flow efficiency.

2. Goal-Based Agent for Healthcare Diagnosis

Introduction

A Goal-Based Agent for Healthcare Diagnosis is an AI agent that selects actions by considering a specific objective or goal. In a healthcare environment, the goal may be to identify the most likely diagnosis, recommend suitable tests, and select an appropriate treatment pathway while considering patient information and clinical rules.

Objectives

- Collect and organize patient symptoms, history, laboratory results, and medical records.
- Set diagnostic goals based on the patient's condition.
- Evaluate possible diagnoses using decision-making logic.
- Recommend suitable diagnostic tests and treatment pathways.
- Support healthcare professionals with consistent data-driven decisions.

System Architecture

The architecture contains a patient-data source, data-preprocessing pipeline, medical knowledge base, goal manager, reasoning engine, decision module, and clinician interface. Patient information is cleaned and transformed before being evaluated by the agent.

Working Principle

7. The system receives patient symptoms, vital signs, medical history, and test results.
8. The data pipeline validates, cleans, and organizes the information.
9. The agent establishes diagnostic goals according to the patient's current condition.
10. Possible diagnoses and treatment options are evaluated against the available evidence.
11. The reasoning engine ranks possible actions according to the defined goal.
12. The selected pathway is presented as decision support for review by a qualified healthcare professional.

Decision-Making Logic

The agent can use decision trees, rule-based reasoning, probabilistic models, or machine-learning models. For example, if a patient's symptoms and test results strongly support a particular condition, the agent can prioritize confirmatory tests and appropriate treatment options. Safety rules can prevent unsuitable actions.

Data Pipeline

Healthcare data may come from electronic health records, laboratory systems, medical devices, and patient inputs. A data pipeline performs ingestion, validation, normalization, feature extraction, analysis, and storage. Access control and privacy protection are essential throughout the pipeline.

Advantages

- Structured and consistent evaluation of patient data.
- Faster identification of relevant diagnostic possibilities.
- Support for evidence-based treatment planning.
- Ability to integrate multiple healthcare data sources.
- Reduced repetitive workload for healthcare professionals.

Limitations and Safety

The agent should be treated as a clinical decision-support system rather than an independent replacement for medical professionals. Data quality, model bias, explainability, privacy, and clinical validation must be addressed before deployment.

Conclusion

A goal-based healthcare agent connects patient information with explicit diagnostic objectives. By combining data pipelines with reasoning and decision-making logic, it can assist clinicians in evaluating diagnostic and treatment pathways in a structured manner.

3. Utility-Based Agent for E-Commerce Recommendations

Introduction

A Utility-Based Agent for E-Commerce Recommendations selects products by estimating how useful or desirable each option is to a particular user. Instead of simply matching products to a query, the agent assigns utility values using factors such as price, relevance, ratings, purchase history, preferences, and product availability.

Objectives

- Understand individual customer preferences.
- Calculate utility scores for available products.
- Rank products according to expected user satisfaction.
- Process large-scale customer and product data efficiently.
- Continuously improve recommendations from user interactions.

System Architecture

The architecture includes user-data sources, product databases, a feature-processing layer, a utility-calculation engine, a recommendation-ranking module, and the e-commerce application. Large-scale data processing frameworks can prepare features from millions of interactions.

Utility Calculation

A simplified utility function can combine several factors: $Utility = w_1(Relevance) + w_2(Rating) + w_3(Preference\ Match) + w_4(Value\ for\ Money) + w_5(Availability)$. The weights can be adjusted according to business objectives and user behavior.

Working Principle

13. Collect user interactions such as searches, clicks, views, purchases, and ratings.
14. Create user and product features from the collected data.
15. Generate a candidate set of products.
16. Calculate a utility score for each candidate.
17. Rank products from highest to lowest expected utility.
18. Display personalized recommendations and collect new feedback.

Large-Scale Data Processing

E-commerce platforms generate large volumes of transactional and behavioral data. Distributed technologies such as Apache Spark, Hadoop, or cloud data-processing services can process this information. Batch processing can train recommendation models, while real-time pipelines can update recommendations based on recent interactions.

Advantages

- Personalized shopping experience.
- Improved relevance of product recommendations.
- Efficient ranking of large product catalogs.
- Better customer engagement and potential sales.
- Ability to combine multiple user-satisfaction factors.

Conclusion

A utility-based recommendation agent makes decisions by comparing the expected usefulness of different products. Large-scale data processing enables the agent to handle massive product catalogs and customer interactions while continuously optimizing recommendations.

4. Learning Agent for Personalized Education System

Introduction

A Learning Agent in a personalized education system improves its behavior by learning from student performance and feedback. It observes how a learner interacts with educational content, identifies strengths and weaknesses, and adapts the difficulty, sequence, and type of learning material.

Objectives

- Track student learning behavior and performance.
- Identify knowledge gaps and learning patterns.
- Adapt content difficulty to individual learners.
- Use feedback to improve recommendations over time.
- Process large educational datasets using distributed technologies.

System Architecture

The system consists of a student interface, learning-data collector, distributed processing layer, learner model, recommendation engine, feedback module, and content repository. Apache Spark or PySpark can process large numbers of student interactions and assessment records.

Working Principle

19. The system collects quiz scores, completion time, attempts, clicks, and feedback.
20. The learning-data pipeline processes and summarizes student activity.
21. The learner model estimates the student's current knowledge and difficulty level.
22. The agent selects suitable lessons, examples, exercises, or assessments.
23. The student interacts with the recommended content.
24. New performance data is fed back into the learning agent so that future recommendations improve.

Adaptation Strategy

If a student repeatedly answers questions incorrectly, the agent can reduce difficulty, provide prerequisite lessons, or offer additional examples. If performance is consistently high, it can increase difficulty and introduce advanced topics. This creates an individualized learning path.

Role of PySpark

PySpark provides a distributed processing environment for analyzing large educational datasets. It can be used for data cleaning, aggregation, feature engineering, model preparation, and large-scale analytics. This allows the platform to process information from many learners efficiently.

Advantages

- Personalized learning experiences.
- Automatic adaptation to student performance.
- Early identification of learning gaps.
- Scalable processing for large numbers of students.
- Continuous improvement through feedback.

Conclusion

A learning agent transforms an educational platform from a fixed-content system into an adaptive environment. By combining feedback, learner modeling, recommendation logic, and PySpark-based distributed processing, the system can provide more personalized learning experiences.

5. Multi-Agent System for Disaster Management

Introduction

A Multi-Agent System (MAS) for Disaster Management uses multiple autonomous agents that cooperate to detect disasters, analyze information, communicate with other agents, and coordinate response activities.

Different agents can perform specialized tasks while sharing information to achieve a common emergency-response objective.

Objectives

- Detect disasters quickly using sensor and external data.
- Share information between specialized agents.
- Coordinate rescue, evacuation, and resource-allocation decisions.
- Support real-time response to rapidly changing conditions.
- Improve reliability through distributed decision-making.

Types of Agents

- Sensor Agents: Collect information from temperature, smoke, water-level, vibration, GPS, camera, and other sensors.
- Decision Agents: Analyze incoming information, estimate risk, and select response actions.
- Communication Agents: Transfer alerts and status information between agents and control centers.
- Resource Agents: Track emergency vehicles, medical supplies, shelters, and rescue teams.
- Evacuation Agents: Help identify safe routes and manage evacuation information.

System Architecture

The MAS contains distributed sensor nodes, communication infrastructure, agent-processing modules, a shared data layer, and a disaster-management control center. Agents can operate independently but communicate through defined protocols to coordinate actions.

Working Principle

25. Sensor agents continuously collect environmental and situational data.
26. Communication agents transmit important events to other agents and the control center.
27. Decision agents combine sensor observations with maps, historical data, and current reports.
28. The system identifies the type and severity of the disaster.
29. Agents coordinate resource allocation, evacuation, and rescue decisions.
30. Updated information is continuously shared so that decisions can change as conditions evolve.

Coordination Strategies

- Message passing: Agents exchange alerts, observations, and commands.
- Shared blackboard/data store: Agents access a common representation of the current situation.
- Negotiation: Agents coordinate resource allocation when multiple areas need assistance.
- Priority-based coordination: Critical incidents receive higher response priority.
- Consensus: Agents compare observations before confirming important decisions.

Real-Time Decision-Making

Real-time processing is essential because disaster conditions can change rapidly. Stream-processing systems can analyze incoming sensor events, detect abnormal patterns, and trigger agent actions. Edge processing can also reduce response time by analyzing critical information close to the data source.

Advantages

- Distributed and scalable disaster-response architecture.
- Faster detection and response.
- Specialized agents can work simultaneously.
- Improved information sharing and coordination.
- Greater flexibility when conditions change.

Conclusion

A Multi-Agent System provides a flexible architecture for disaster management because multiple specialized agents can cooperate while operating independently. By combining real-time sensing, communication, distributed processing, and coordinated decision-making, the system can support faster and more effective emergency response.

Overall Conclusion

Intelligent agents provide different approaches for solving real-world problems. Reactive agents respond immediately to changing conditions, goal-based agents select actions that achieve defined objectives, utility-based agents optimize decisions according to user satisfaction, learning agents improve through feedback, and multi-agent systems coordinate multiple autonomous components. Together, these agent architectures demonstrate how artificial intelligence and data-processing technologies can support smart, scalable, and adaptive systems.